Graph Data Structure Study Guide: BFS and DFS

1. Breadth First Search (BFS)

1.1 What is BFS?

Breadth First Search (BFS) is a graph traversal algorithm that explores vertices level by level. It visits all vertices at the current depth before moving to vertices at the next depth level. BFS uses a **queue** data structure to keep track of vertices to be explored.

1.2 How BFS Works

1. Start from a given source vertex
2. Mark the source as visited and add it to the queue
3. While the queue is not empty:
    - Remove a vertex from the front of the queue
    - Visit all unvisited neighbors of this vertex
    - Mark each unvisited neighbor as visited and add to queue

1.3 BFS Algorithm Properties

- **Time Complexity:** O(V + E) where V is vertices and E is edges
- **Space Complexity:** O(V) for the queue and visited array
- **Complete:** Always finds a solution if one exists
- **Optimal:** Finds shortest path in unweighted graphs

1.4 BFS Implementation

java

```java
import java.util.*;

public class BFSGraph {
    private int vertices;
    private ArrayList<ArrayList<Integer>> adjacencyList;

    public BFSGraph(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>();

        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    // Add edge to the graph
    public void addEdge(int source, int destination) {
        adjacencyList.get(source).add(destination);
        adjacencyList.get(destination).add(source); // For undirected graph
    }

    // BFS traversal starting from a given vertex
    public void BFS(int startVertex) {
        boolean[] visited = new boolean[vertices];
        Queue<Integer> queue = new LinkedList<>();

        // Mark start vertex as visited and enqueue it
        visited[startVertex] = true;
        queue.offer(startVertex);

        System.out.println("BFS Traversal starting from vertex " + startVertex + ":");

        while (!queue.isEmpty()) {
            // Dequeue a vertex and print it
            int currentVertex = queue.poll();
            System.out.print(currentVertex + " ");

            // Get all adjacent vertices of currentVertex
            for (int neighbor : adjacencyList.get(currentVertex)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.offer(neighbor);
                }
            }
        }
        System.out.println();
```

```java
}

// BFS to find shortest path between two vertices
public int shortestPath(int start, int end) {
    if (start == end) return 0;

    boolean[] visited = new boolean[vertices];
    Queue<Integer> queue = new LinkedList<>();
    int[] distance = new int[vertices];

    visited[start] = true;
    queue.offer(start);
    distance[start] = 0;

    while (!queue.isEmpty()) {
        int currentVertex = queue.poll();

        for (int neighbor : adjacencyList.get(currentVertex)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                distance[neighbor] = distance[currentVertex] + 1;
                queue.offer(neighbor);

                if (neighbor == end) {
                    return distance[neighbor];
                }
            }
        }
    }

    return -1; // Path not found
}

// BFS to check if graph is connected
public boolean isConnected() {
    boolean[] visited = new boolean[vertices];
    Queue<Integer> queue = new LinkedList<>();

    visited[0] = true;
    queue.offer(0);
    int visitedCount = 1;

    while (!queue.isEmpty()) {
        int currentVertex = queue.poll();

        for (int neighbor : adjacencyList.get(currentVertex)) {
            if (!visited[neighbor]) {
```

```
                    visited[neighbor] = true;
                    queue.offer(neighbor);
                    visitedCount++;
                }
            }
        }

        return visitedCount == vertices;
    }
}
```

1.5 BFS Applications

- **Shortest Path:** Finding shortest path in unweighted graphs
- **Level Order Traversal:** Tree level-by-level exploration
- **Connected Components:** Finding all connected components
- **Bipartite Graph Check:** Determining if graph is bipartite
- **Web Crawling:** Exploring web pages level by level
- **Social Networking:** Finding connections within certain degrees

1.6 BFS Example Execution

```java
public class BFSExample {
    public static void main(String[] args) {
        BFSGraph graph = new BFSGraph(6);

        // Adding edges
        graph.addEdge(0, 1);
        graph.addEdge(0, 2);
        graph.addEdge(1, 3);
        graph.addEdge(1, 4);
        graph.addEdge(2, 5);

        // Perform BFS
        graph.BFS(0);

        // Find shortest path
        System.out.println("Shortest path from 0 to 5: " +
                           graph.shortestPath(0, 5));

        // Check connectivity
        System.out.println("Is graph connected? " +
                           graph.isConnected());
    }
}
```

2. Depth First Search (DFS)

2.1 What is DFS?

Depth First Search (DFS) is a graph traversal algorithm that explores as far as possible along each branch before backtracking. DFS uses a **stack** data structure (or recursion which uses the call stack) to keep track of vertices to be explored.

2.2 How DFS Works

1. Start from a given source vertex
2. Mark the source as visited
3. For each unvisited neighbor of the current vertex:
    - Recursively apply DFS to the neighbor
4. Backtrack when no more unvisited neighbors exist

2.3 DFS Algorithm Properties

- **Time Complexity:** O(V + E) where V is vertices and E is edges

- **Space Complexity:** O(V) for the recursion stack and visited array
- **Complete:** May not find shortest path
- **Memory Efficient:** Uses less memory than BFS for sparse graphs

2.4 DFS Implementation

java

```java
import java.util.*;

public class DFSGraph {
    private int vertices;
    private ArrayList<ArrayList<Integer>> adjacencyList;

    public DFSGraph(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>();

        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    // Add edge to the graph
    public void addEdge(int source, int destination) {
        adjacencyList.get(source).add(destination);
        adjacencyList.get(destination).add(source); // For undirected graph
    }

    // Recursive DFS traversal
    public void DFS(int startVertex) {
        boolean[] visited = new boolean[vertices];
        System.out.println("DFS Traversal starting from vertex " + startVertex + ":");
        DFSUtil(startVertex, visited);
        System.out.println();
    }

    // Utility function for recursive DFS
    private void DFSUtil(int vertex, boolean[] visited) {
        visited[vertex] = true;
        System.out.print(vertex + " ");

        // Recursively visit all unvisited neighbors
        for (int neighbor : adjacencyList.get(vertex)) {
            if (!visited[neighbor]) {
                DFSUtil(neighbor, visited);
            }
        }
    }

    // Iterative DFS using explicit stack
    public void DFSIterative(int startVertex) {
        boolean[] visited = new boolean[vertices];
        Stack<Integer> stack = new Stack<>();
```

```java
        stack.push(startVertex);
        System.out.println("Iterative DFS starting from vertex " + startVertex + ":");

        while (!stack.isEmpty()) {
            int currentVertex = stack.pop();

            if (!visited[currentVertex]) {
                visited[currentVertex] = true;
                System.out.print(currentVertex + " ");

                // Add all unvisited neighbors to stack
                for (int neighbor : adjacencyList.get(currentVertex)) {
                    if (!visited[neighbor]) {
                        stack.push(neighbor);
                    }
                }
            }
        }
        System.out.println();
    }

    // DFS to detect cycle in undirected graph
    public boolean hasCycle() {
        boolean[] visited = new boolean[vertices];

        for (int i = 0; i < vertices; i++) {
            if (!visited[i]) {
                if (hasCycleUtil(i, visited, -1)) {
                    return true;
                }
            }
        }
        return false;
    }

    private boolean hasCycleUtil(int vertex, boolean[] visited, int parent) {
        visited[vertex] = true;

        for (int neighbor : adjacencyList.get(vertex)) {
            if (!visited[neighbor]) {
                if (hasCycleUtil(neighbor, visited, vertex)) {
                    return true;
                }
            } else if (neighbor != parent) {
                return true; // Back edge found, cycle exists
            }
```

```java
        }
        return false;
    }

    // DFS to find connected components
    public void findConnectedComponents() {
        boolean[] visited = new boolean[vertices];
        int componentCount = 0;

        System.out.println("Connected Components:");
        for (int i = 0; i < vertices; i++) {
            if (!visited[i]) {
                System.out.print("Component " + (++componentCount) + ": ");
                DFSUtil(i, visited);
                System.out.println();
            }
        }
    }

    // DFS to check if path exists between two vertices
    public boolean hasPath(int start, int end) {
        if (start == end) return true;

        boolean[] visited = new boolean[vertices];
        return hasPathUtil(start, end, visited);
    }

    private boolean hasPathUtil(int current, int target, boolean[] visited) {
        visited[current] = true;

        if (current == target) return true;

        for (int neighbor : adjacencyList.get(current)) {
            if (!visited[neighbor]) {
                if (hasPathUtil(neighbor, target, visited)) {
                    return true;
                }
            }
        }
        return false;
    }
}
```

2.5 DFS Applications

- **Cycle Detection:** Finding cycles in graphs

- **Topological Sorting:** Ordering vertices in directed acyclic graphs
- **Connected Components:** Finding strongly connected components
- **Path Finding:** Checking if path exists between vertices
- **Maze Solving:** Finding exit in maze problems
- **Backtracking Problems:** N-Queens, Sudoku solver

2.6 DFS Example Execution

```java
public class DFSExample {
    public static void main(String[] args) {
        DFSGraph graph = new DFSGraph(7);

        // Adding edges to create a graph with cycle
        graph.addEdge(0, 1);
        graph.addEdge(0, 2);
        graph.addEdge(1, 3);
        graph.addEdge(1, 4);
        graph.addEdge(2, 5);
        graph.addEdge(3, 4); // Creates a cycle
        graph.addEdge(6, 6); // Isolated vertex

        // Perform DFS
        graph.DFS(0);
        graph.DFSIterative(0);

        // Check for cycle
        System.out.println("Graph has cycle: " + graph.hasCycle());

        // Find connected components
        graph.findConnectedComponents();

        // Check path existence
        System.out.println("Path exists from 0 to 5: " +
                           graph.hasPath(0, 5));
        System.out.println("Path exists from 0 to 6: " +
                           graph.hasPath(0, 6));
    }
}
```

2.7 BFS vs DFS Comparison

Aspect	BFS	DFS
Data Structure	Queue	Stack (or Recursion)
Memory Usage	Higher (stores all nodes at current level)	Lower (stores path from root)
Shortest Path	Yes (unweighted graphs)	No
Complete	Yes	Yes
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V)	O(V)
Applications	Shortest path, level-order traversal	Cycle detection, topological sort

2.8 Advanced DFS: Topological Sort

java

```java
public class TopologicalSort {
    private int vertices;
    private ArrayList<ArrayList<Integer>> adjacencyList;

    public TopologicalSort(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>();

        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination) {
        adjacencyList.get(source).add(destination); // Directed graph
    }

    public void topologicalSort() {
        Stack<Integer> stack = new Stack<>();
        boolean[] visited = new boolean[vertices];

        // Call DFS for all unvisited vertices
        for (int i = 0; i < vertices; i++) {
            if (!visited[i]) {
                topologicalSortUtil(i, visited, stack);
            }
        }

        // Print topological order
        System.out.println("Topological Sort:");
        while (!stack.isEmpty()) {
            System.out.print(stack.pop() + " ");
        }
        System.out.println();
    }

    private void topologicalSortUtil(int vertex, boolean[] visited,
                                     Stack<Integer> stack) {
        visited[vertex] = true;

        // Recursively call for all adjacent vertices
        for (int neighbor : adjacencyList.get(vertex)) {
            if (!visited[neighbor]) {
                topologicalSortUtil(neighbor, visited, stack);
            }
        }
```

```
        // Push current vertex to stack after visiting all neighbors
        stack.push(vertex);
    }
}
```

Quiz Questions

Question 1: Which data structure does BFS use internally for traversal? a) Stack b) Queue c) Array d) Linked List

Correct Answer: b) Queue

Question 2: What is the time complexity of both BFS and DFS for a graph with V vertices and E edges? a) O(V) b) O(E) c) O(V + E) d) O(V × E)

Correct Answer: c) O(V + E)

Question 3: Which algorithm is better for finding the shortest path in an unweighted graph? a) DFS b) BFS c) Both are equally good d) Neither can find shortest path

Correct Answer: b) BFS

Question 4: In DFS, when do we backtrack to the previous vertex? a) When we visit a vertex b) When we find the target vertex c) When we encounter a visited vertex d) When there are no more unvisited neighbors

Correct Answer: d) When there are no more unvisited neighbors

Question 5: Which algorithm is more suitable for detecting cycles in an undirected graph? a) BFS only b) DFS only c) Both BFS and DFS d) Neither BFS nor DFS

Correct Answer: c) Both BFS and DFS

Practice Coding Questions

Question 1: Implement a method `findAllPaths(int start, int end)` using DFS that finds and prints all possible paths between two given vertices in a graph. The method should handle both directed and undirected graphs.

Question 2: Write a Java program using BFS to solve the "Word Ladder" problem. Given two words (start and end) and a dictionary, find the shortest transformation sequence from start to end where each transformation changes exactly one letter and each intermediate word must exist in the dictionary.

Question 3: Implement a method `isBipartite()` using BFS that determines whether a given undirected graph is bipartite (can be colored with two colors such that no two adjacent vertices have the same color).

Return true if the graph is bipartite, false otherwise.

Question 4: Create a class `GraphClone` with a method `cloneGraph(Node node)` using DFS that creates a deep copy of an undirected graph. Each node contains a value and a list of neighbors. Return the cloned graph starting from the given node.

Question 5: Write a method `findLongestPath(int start)` using DFS that finds the longest simple path (without cycles) starting from a given vertex in an unweighted directed acyclic graph (DAG). Return both the length of the longest path and the actual path as a list of vertices.